RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Department of Artificial Intelligence and Data Science

V Semester B. Tech AIDS

AD23532 – Principles of Data Science

Academic Year: 2026-2027 **Batch:** 2024-2028

Name: Praveen L

Ex 6a: Exploratory Data Analysis

Register Number: 241801213 **Date:** 02-10-2026

Dataset: Netflix Movies and TV Shows – netflix_titles.csv (Kaggle) **Tools:** Python, Pandas, Matplotlib, Seaborn, WordCloud

Aim

To perform Exploratory Data Analysis on the Netflix titles dataset and use statistical summaries and visualisations to study the content distribution, ratings, release and addition trends, durations, countries, genres, directors, actors and descriptions of the Netflix library.

Task 1: Load the Netflix titles dataset into a DataFrame

In [1]:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from collections import Counter
from wordcloud import WordCloud, STOPWORDS

import warnings
warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid')
pd.set_option('display.max_colwidth', 45)

df = pd.read_csv('netflix_titles.csv')
print("Dataset loaded successfully")
print("Shape:", df.shape)
```

Out [1]:

```
Dataset loaded successfully
Shape: (8807, 12)
```

Task 2: Explore the dataset

In [2]:

```
df.head()
```

Out [2]:

Columns 1–6 of 12

	show_id	type	title	director	cast	country
0	s1	Movie	Dick Johnson Is Dead	Kirsten Johnson	NaN	United States
1	s2	TV Show	Blood & Water	NaN	Ama Qamata, Khosi Ngema, Gail Mabalane, Th...	South Africa
2	s3	TV Show	Ganglands	Julien Leclercq	Sami Bouajila, Tracy Gotoas, Samuel Jouy, ...	NaN
3	s4	TV Show	Jailbirds New Orleans	NaN	NaN	NaN
4	s5	TV Show	Kota Factory	NaN	Mayur More, Jitendra Kumar, Ranjan Raj, Al...	India

Columns 7–11 of 12

	date_added	release_year	rating	duration	listed_in
0	September 25, 2021	2020	PG-13	90 min	Documentaries
1	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, TV Dramas, TV Myst...
2	September 24, 2021	2021	TV-MA	1 Season	Crime TV Shows, International TV Shows, TV...
3	September 24, 2021	2021	TV-MA	1 Season	Docuseries, Reality TV
4	September 24, 2021	2021	TV-MA	2 Seasons	International TV Shows, Romantic TV Shows,...

Columns 12–12 of 12

	description
0	As her father nears the end of his life, f...
1	After crossing paths at a party, a Cape To...
2	To protect his family from a powerful drug...
3	Feuds, flirtations and toilet talk go down...
4	In a city of coaching centers known to tra...

5 rows × 12 columns

In [3]:

```
df.tail()
```

Out [3]:

Columns 1–6 of 12

	show_id	type	title	director	cast	country
8802	s8803	Movie	Zodiac	David Fincher	Mark Ruffalo, Jake Gyllenhaal, Robert Down...	United States
8803	s8804	TV Show	Zombie Dumb	NaN	NaN	NaN
8804	s8805	Movie	Zombieland	Ruben Fleischer	Jesse Eisenberg, Woody Harrelson, Emma Sto...	United States
8805	s8806	Movie	Zoom	Peter Hewitt	Tim Allen, Courteney Cox, Chevy Chase, Kat...	United States
8806	s8807	Movie	Zubaan	Mozez Singh	Vicky Kaushal, Sarah-Jane Dias, Raaghav Ch...	India

Columns 7–11 of 12

	date_added	release_year	rating	duration	listed_in
8802	November 20, 2019	2007	R	158 min	Cult Movies, Dramas, Thrillers
8803	July 1, 2019	2018	TV-Y7	2 Seasons	Kids' TV, Korean TV Shows, TV Comedies
8804	November 1, 2019	2009	R	88 min	Comedies, Horror Movies
8805	January 11, 2020	2006	PG	88 min	Children & Family Movies, Comedies
8806	March 2, 2019	2015	TV-14	111 min	Dramas, International Movies, Music & Musi...

Columns 12–12 of 12

	description
8802	A political cartoonist, a crime reporter a...
8803	While living alone in a spooky town, a you...
8804	Looking to survive in a world taken over b...
8805	Dragged from civilian life, a former super...
8806	A scrappy but poor boy worms his way into ...

5 rows × 12 columns

In [4]:

```
df.info()
```

Out [4]:

```
<class 'pandas.DataFrame'>
RangeIndex: 8807 entries, 0 to 8806
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   show_id       8807 non-null   str
 1   type          8807 non-null   str
 2   title         8807 non-null   str
 3   director      6173 non-null   str
 4   cast          7982 non-null   str
 5   country       7976 non-null   str
 6   date_added    8797 non-null   str
 7   release_year  8807 non-null   int64
 8   rating        8803 non-null   str
 9   duration      8804 non-null   str
 10  listed_in     8807 non-null   str
 11  description   8807 non-null   str
dtypes: int64(1), str(11)
memory usage: 825.8 KB
```

In [5]:

```
missing = pd.DataFrame({'Missing Values': df.isnull().sum(),
                        'Missing (%)': (df.isnull().mean() * 100).round(2),
                        'Unique Values': df.nunique()})
print("Duplicate rows:", df.duplicated().sum())
missing
```

Out [5]:

```
Duplicate rows: 0
```

	Missing Values	Missing (%)	Unique Values
show_id	0	0.00	8807
type	0	0.00	2
title	0	0.00	8807
director	2634	29.91	4528
cast	825	9.37	7692
country	831	9.44	748
date_added	10	0.11	1767
release_year	0	0.00	74
rating	4	0.05	17
duration	3	0.03	220
listed_in	0	0.00	514
description	0	0.00	8775

12 rows × 3 columns

In [6]:

```
df.describe(include='all').T[['count', 'unique', 'top', 'freq']]
```

Out [6]:

	count	unique	top	freq
show_id	8807.0	8807.0	s1	1.0
type	8807.0	2.0	Movie	6131.0
title	8807.0	8807.0	Dick Johnson Is Dead	1.0
director	6173.0	4528.0	Rajiv Chilaka	19.0
cast	7982.0	7692.0	David Attenborough	19.0
country	7976.0	748.0	United States	2818.0
date_added	8797.0	1767.0	January 1, 2020	109.0
release_year	8807.0	NaN	NaN	NaN
rating	8803.0	17.0	TV-MA	3207.0
duration	8804.0	220.0	1 Season	1793.0
listed_in	8807.0	514.0	Dramas, International Movies	362.0
description	8807.0	8775.0	Paranormal activity at a lush, abandoned p...	4.0

12 rows × 4 columns

Data cleaning

In [7]:

```
# 1. Three rows have the movie duration stored in the rating column by mistake
wrong_rating = df['rating'].str.contains('min', na=False)
print("Rows with duration in the rating column:")
print(df.loc[wrong_rating, ['title', 'rating', 'duration']].to_string(), "\n")
df.loc[wrong_rating, 'duration'] = df.loc[wrong_rating, 'rating']
df.loc[wrong_rating, 'rating'] = np.nan

# 2. Fill missing text values
for col in ['director', 'cast', 'country']:
    df[col] = df[col].fillna('Unknown')
df['rating'] = df['rating'].fillna(df['rating'].mode()[0])

# 3. Convert date_added to datetime and create year and month columns
df['date_added'] = pd.to_datetime(df['date_added'].str.strip(), format='%B %d, %Y')
df['year_added'] = df['date_added'].dt.year
df['month_added'] = df['date_added'].dt.month_name()

# 4. Split duration into a number and a unit (min or Season)
df['duration_value'] = df['duration'].str.extract(r'(\d+)').astype(float)
df['duration_unit'] = np.where(df['duration'].str.contains('Season'), 'Seasons', 'Minutes')

print("Missing values after cleaning:")
print(df[['director', 'cast', 'country', 'rating', 'duration', 'date_added']].isnull().sum())
```

Out [7]:

```
Rows with duration in the rating column:
                                     title  rating duration
5541                     Louis C.K. 2017  74 min      NaN
5794                 Louis C.K.: Hilarious  84 min      NaN
5813  Louis C.K.: Live at the Comedy Store  66 min      NaN

Missing values after cleaning:
director       0
cast           0
country        0
rating         0
duration       0
date_added    10
dtype: int64
```

> **Observation:** The dataset has 8,807 titles and 12 columns with no duplicate rows. The *director* column has the most missing values (29.91%), followed by *country* (9.44%) and *cast* (9.37%). Three Louis C.K. specials had their duration stored in the *rating* column; these were moved to *duration*. The 10 titles without a *date_added* are left out only from the date-based analysis.

Task 3: Content Distribution Analysis

3(a) How many Movies vs TV Shows are in the dataset?

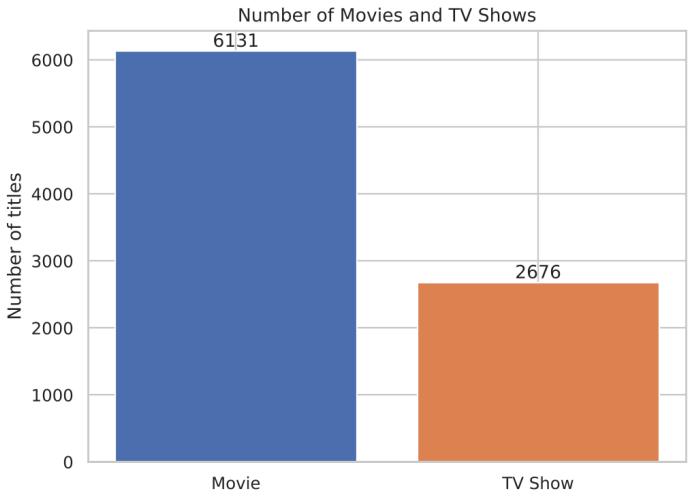
In [8]:

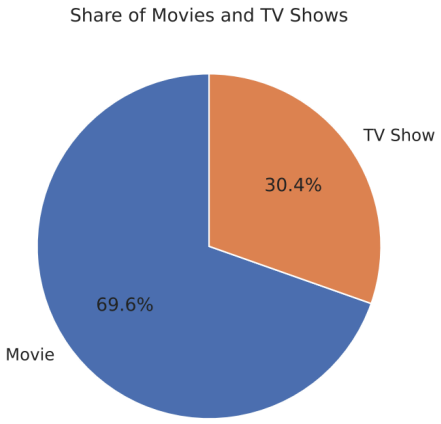
```
type_counts = df['type'].value_counts()
print(pd.DataFrame({'Count': type_counts, 'Percentage (%)': (type_counts / len(df) *
100).round(2)}))

type_colors = {'Movie': '#4C72B0', 'TV Show': '#DD8452'}
fig, axes = plt.subplots(1, 2, figsize=(12, 4.8))
bars = axes[0].bar(type_counts.index, type_counts.values, color=[type_colors[t] for t in
type_counts.index])
axes[0].bar_label(bars)
axes[0].set_title('Number of Movies and TV Shows')
axes[0].set_ylabel('Number of titles')
axes[1].pie(type_counts, labels=type_counts.index, autopct='%1.1f%%', startangle=90,
            colors=[type_colors[t] for t in type_counts.index], wedgeprops={'edgecolor': 'white'})
axes[1].set_title('Share of Movies and TV Shows')
plt.tight_layout()
plt.show()
```

Out [8]:

```
         Count  Percentage (%)
type
Movie     6131           69.62
TV Show   2676           30.38
```





3(b) What is the distribution of ratings?

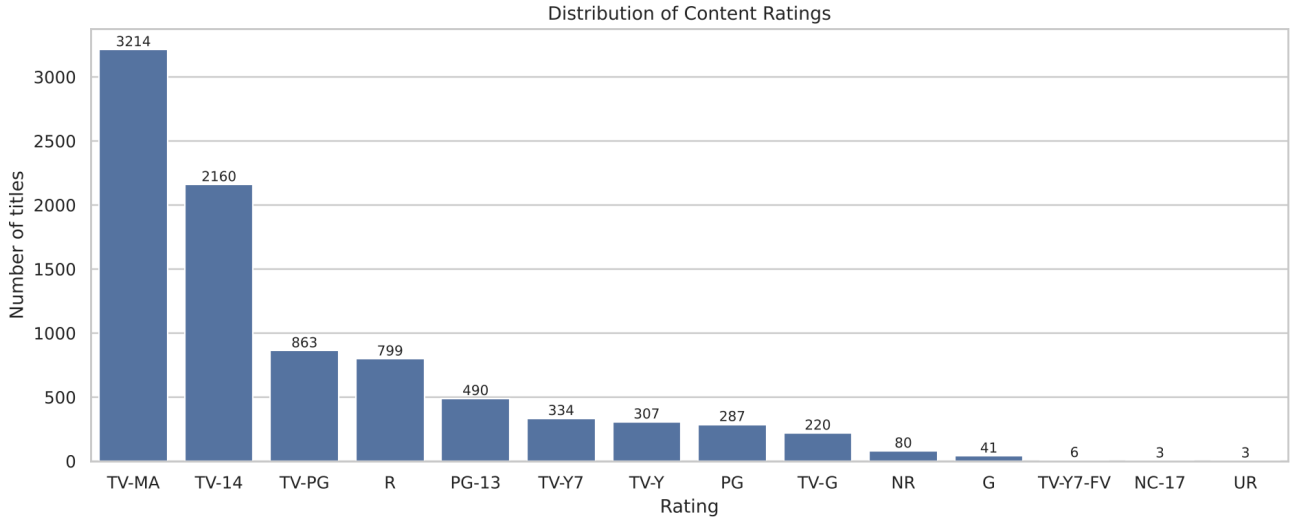
In [9]:

```
rating_counts = df['rating'].value_counts()

plt.figure(figsize=(12, 5))
ax = sns.barplot(x=rating_counts.index, y=rating_counts.values, color='#4C72B0')
ax.bar_label(ax.containers[0], fontsize=9)
plt.title('Distribution of Content Ratings')
plt.xlabel('Rating')
plt.ylabel('Number of titles')
plt.tight_layout()
plt.show()

print((rating_counts / len(df) * 100).round(2).head(5).to_string())
```

Out [9]:




```
rating
TV-MA    36.49
TV-14    24.53
TV-PG     9.80
R         9.07
PG-13     5.56
```

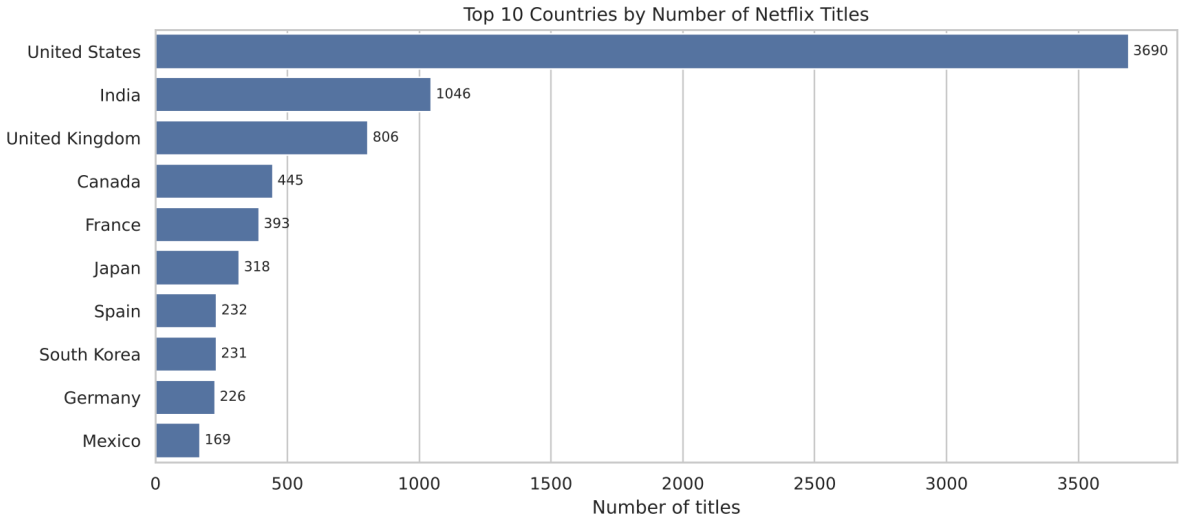
3(c) Which countries have the most Netflix content?

In [10]:

```
# A title can list several countries, so split them and count each country
countries = (df.loc[df['country'] != 'Unknown', 'country']
             .str.split(',').explode().str.strip())
countries = countries[countries != '']
top_countries = countries.value_counts().head(10)

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=top_countries.values, y=top_countries.index, color='#4C72B0')
ax.bar_label(ax.containers[0], padding=3, fontsize=9)
plt.title('Top 10 Countries by Number of Netflix Titles')
plt.xlabel('Number of titles')
plt.ylabel('')
plt.tight_layout()
plt.show()
```

Out [10]:



> **Observation:** Movies make up 69.6% of the library (6,131 titles) and TV shows 30.4% (2,676). Most content is for mature audiences: TV-MA (36.5%) and TV-14 (24.5%) are the most common ratings. The United States produces by far the most content (3,690 titles), followed by India (1,046) and the United Kingdom (806).

Task 4: Time-Based Trend Analysis

4(a) In which years were the most shows/movies released?

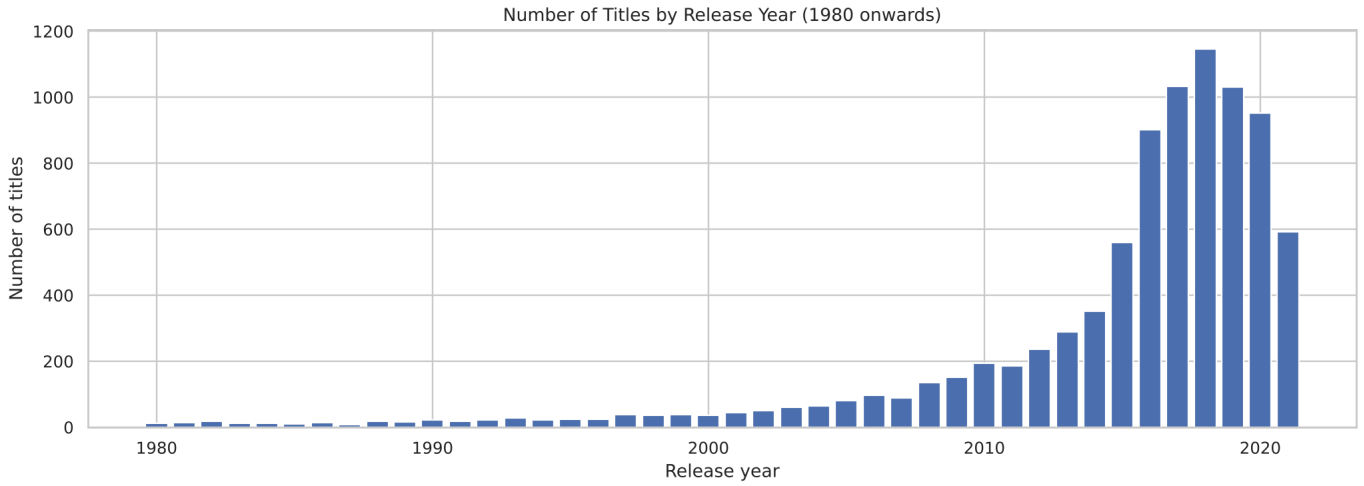
In [11]:

```
release_counts = df['release_year'].value_counts().sort_index()
print("Top 10 release years:")
print(df['release_year'].value_counts().head(10).to_string())

plt.figure(figsize=(13, 4.8))
recent = release_counts[release_counts.index >= 1980]
plt.bar(recent.index, recent.values, color='#4C72B0')
plt.title('Number of Titles by Release Year (1980 onwards)')
plt.xlabel('Release year')
plt.ylabel('Number of titles')
plt.tight_layout()
plt.show()
```

Out [11]:

```
Top 10 release years:
release_year
2018    1147
2017    1032
2019    1030
2020     953
2016     902
2021     592
2015     560
2014     352
2013     288
2012     237
```



4(b) In which years were titles added to Netflix?

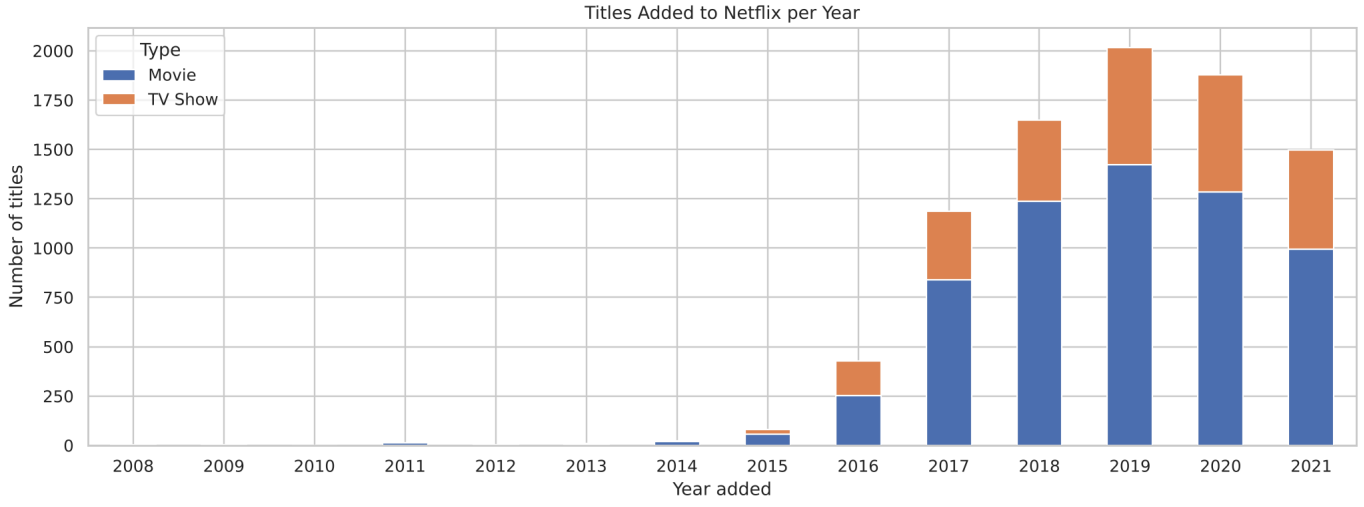
In [12]:

```
added_by_type = df.dropna(subset=['year_added']).groupby(['year_added',
'type']).size().unstack(fill_value=0)
added_by_type.index = added_by_type.index.astype(int)
added_by_type['Total'] = added_by_type.sum(axis=1)
print(added_by_type.T.to_string())

ax = added_by_type[['Movie', 'TV Show']].plot(kind='bar', stacked=True, figsize=(13, 5),
                                              color=[type_colors['Movie'], type_colors['TV Show']])
plt.title('Titles Added to Netflix per Year')
plt.xlabel('Year added')
plt.ylabel('Number of titles')
plt.xticks(rotation=0)
plt.legend(title='Type')
plt.tight_layout()
plt.show()
```

Out [12]:

year_added	2008	2009	2010	2011	2012	2013	2014	2015	2016	2017	2018	2019	2020	2021
type														
Movie	1	2	1	13	3	6	19	56	253	839	1237	1424	1284	993
TV Show	1	0	0	0	0	5	5	26	176	349	412	592	595	505
Total	2	2	1	13	3	11	24	82	429	1188	1649	2016	1879	1498



4(c) How has Netflix's content growth trend changed over time?

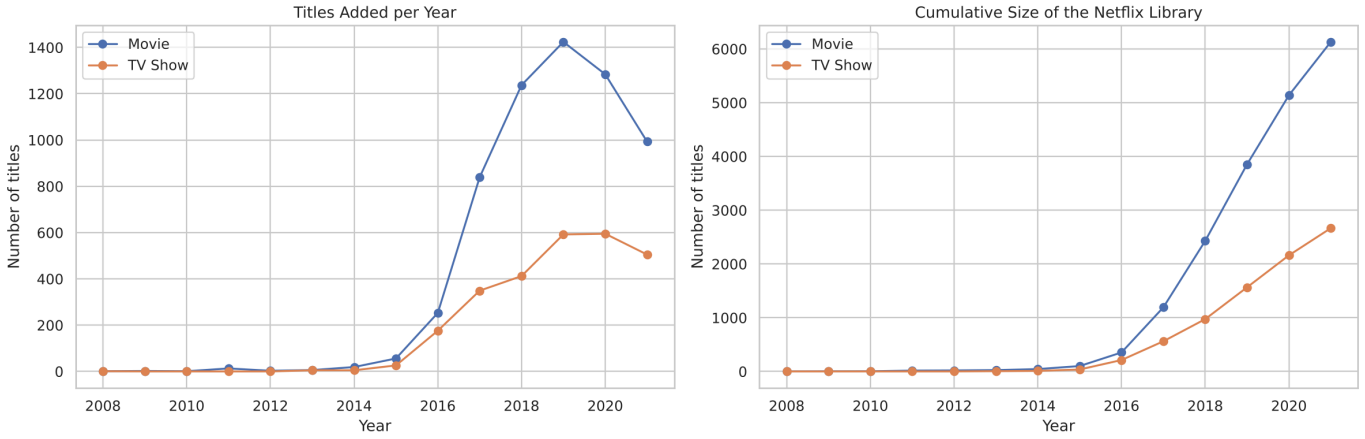
In [13]:

```
cumulative = added_by_type[['Movie', 'TV Show']].cumsum()

fig, axes = plt.subplots(1, 2, figsize=(15, 5))
for t in ['Movie', 'TV Show']:
    axes[0].plot(added_by_type.index, added_by_type[t], marker='o', color=type_colors[t], label=t)
    axes[1].plot(cumulative.index, cumulative[t], marker='o', color=type_colors[t], label=t)
axes[0].set_title('Titles Added per Year')
axes[1].set_title('Cumulative Size of the Netflix Library')
for ax in axes:
    ax.set_xlabel('Year')
    ax.set_ylabel('Number of titles')
    ax.legend()
plt.tight_layout()
plt.show()

growth = added_by_type['Total'].pct_change().mul(100).round(1)
print("Year-on-year growth in titles added (%):")
print(growth.loc[2015:].to_string())
```

Out [13]:




```
Year-on-year growth in titles added (%):
year_added
2015    241.7
2016    423.2
2017    176.9
2018     38.8
2019     22.3
2020     -6.8
2021    -20.3
```

> **Observation:** Most titles were released recently: 2018 has the most releases (1,147), followed by 2017 and 2019. Netflix added very little content before 2015, then the number of titles added each year grew rapidly, peaking at 2,016 titles in 2019. Additions fell in 2020 and 2021 (the 2021 data only runs to September, and production slowed during COVID-19). Since 2018, TV shows have made up a growing share of new additions (25% in 2018 to 34% in 2021).

Task 5: Duration & Content Type Analysis

5(a) What is the distribution of movie durations (in minutes)?

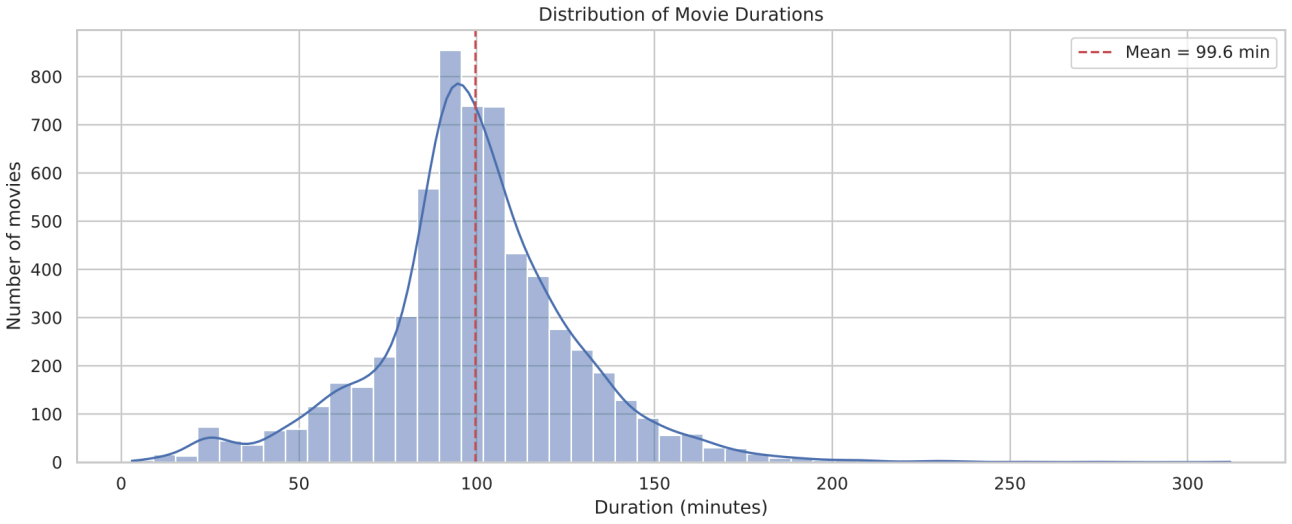
In [14]:

```
movie_duration = df.loc[df['type'] == 'Movie', 'duration_value']
print(movie_duration.describe().round(2).to_string())

plt.figure(figsize=(12, 5))
sns.histplot(movie_duration, bins=50, kde=True, color='#4C72B0')
plt.axvline(movie_duration.mean(), color='#C44E52', linestyle='--',
            label=f'Mean = {movie_duration.mean():.1f} min')
plt.title('Distribution of Movie Durations')
plt.xlabel('Duration (minutes)')
plt.ylabel('Number of movies')
plt.legend()
plt.tight_layout()
plt.show()
```

Out [14]:

```
count    6131.00
mean       99.56
std        28.29
min         3.00
25%        87.00
50%        98.00
75%       114.00
max       312.00
```



5(b) How many TV shows exist per number of seasons?

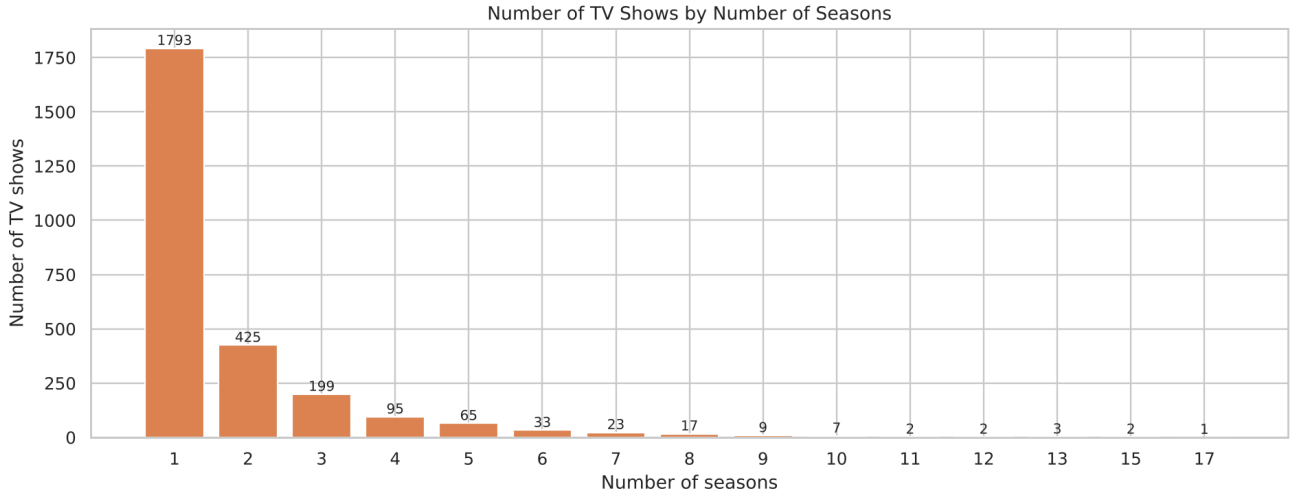
In [15]:

```
seasons = df.loc[df['type'] == 'TV Show', 'duration_value'].astype(int).value_counts().sort_index()
print(seasons.to_string())

plt.figure(figsize=(12, 4.8))
bars = plt.bar(seasons.index.astype(str), seasons.values, color='#DD8452')
plt.bar_label(bars, fontsize=9)
plt.title('Number of TV Shows by Number of Seasons')
plt.xlabel('Number of seasons')
plt.ylabel('Number of TV shows')
plt.tight_layout()
plt.show()
```

Out [15]:

duration_value	
1	1793
2	425
3	199
4	95
5	65
6	33
7	23
8	17
9	9
10	7
11	2
12	2
13	3
15	2
17	1



> **Observation:** Most movies last between 87 and 114 minutes (the middle 50%), with an average of about 100 minutes; 124 short films last under 30 minutes and the longest movie lasts 312 minutes. Most TV shows are short: 1,793 of the 2,676 shows (67%) have only one season, and very few have more than five seasons.

Task 6: Genre & Categories Analysis

6(a) What are the top 10 most common genres on Netflix?

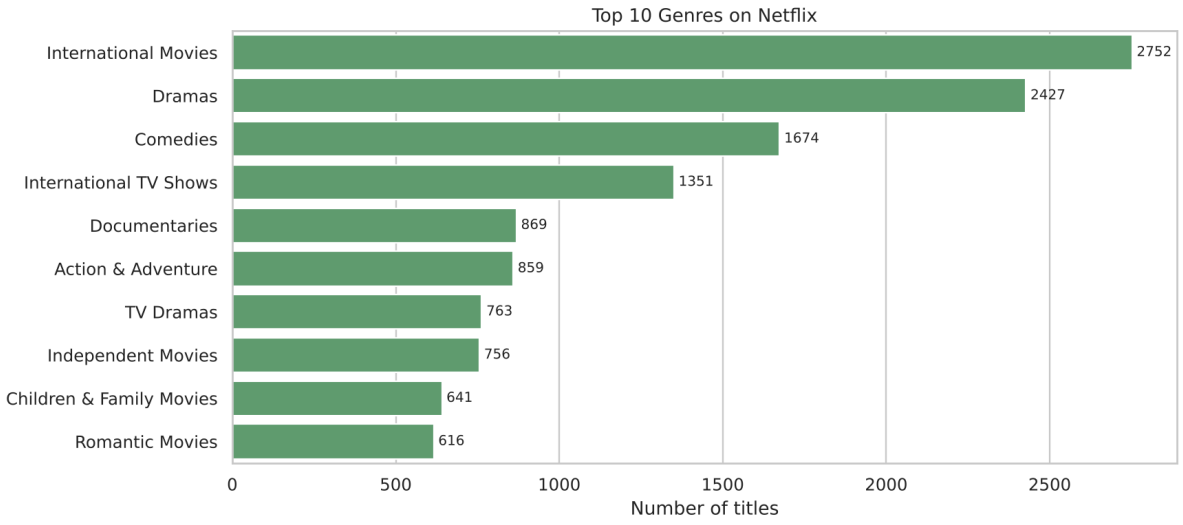
In [16]:

```
genres = df['listed_in'].str.split(',').explode().str.strip()
top_genres = genres.value_counts().head(10)
print("Number of different genres:", genres.nunique())

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=top_genres.values, y=top_genres.index, color='#55A868')
ax.bar_label(ax.containers[0], padding=3, fontsize=9)
plt.title('Top 10 Genres on Netflix')
plt.xlabel('Number of titles')
plt.ylabel('')
plt.tight_layout()
plt.show()
```

Out [16]:

```
Number of different genres: 42
```



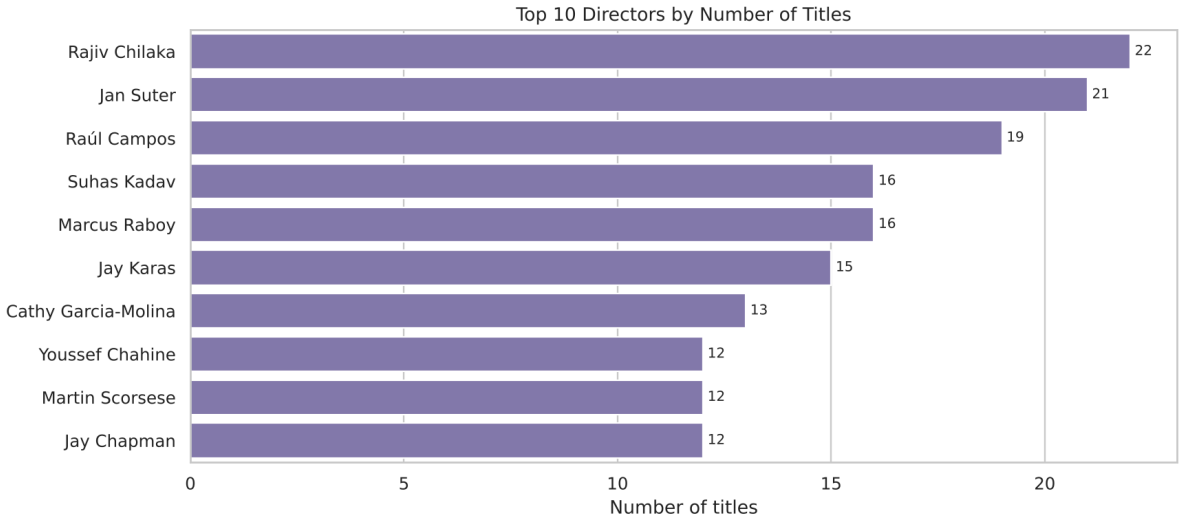
6(b) Which directors have the most titles on Netflix?

In [17]:

```
directors = (df.loc[df['director'] != 'Unknown', 'director']
             .str.split(',').explode().str.strip())
top_directors = directors.value_counts().head(10)

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=top_directors.values, y=top_directors.index, color='#8172B3')
ax.bar_label(ax.containers[0], padding=3, fontsize=9)
plt.title('Top 10 Directors by Number of Titles')
plt.xlabel('Number of titles')
plt.ylabel('')
plt.tight_layout()
plt.show()
```

Out [17]:



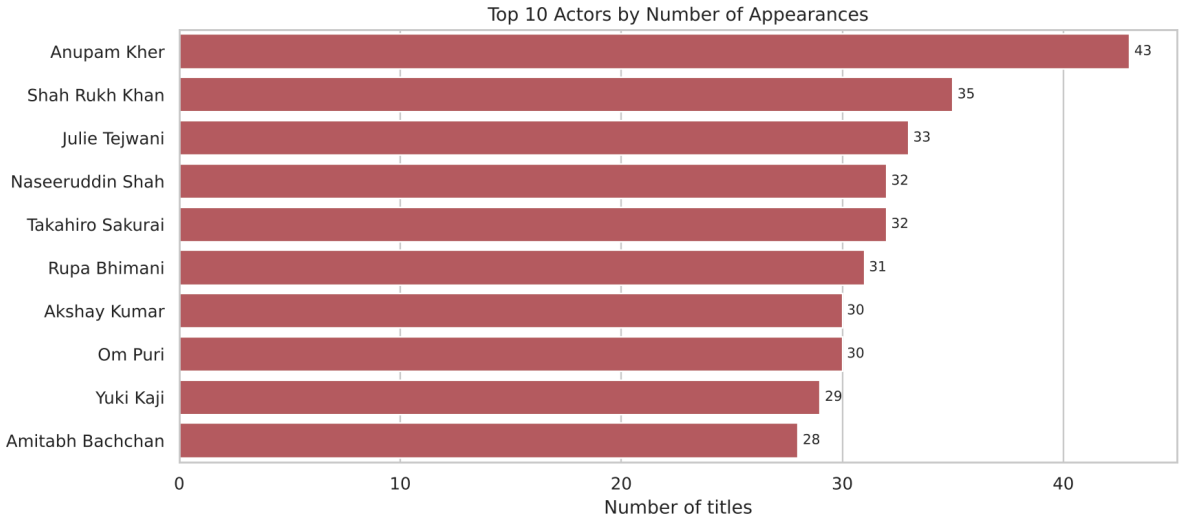
6(c) Which actors appear most frequently?

In [18]:

```
actors = df.loc[df['cast'] != 'Unknown', 'cast'].str.split(',').explode().str.strip()
top_actors = actors.value_counts().head(10)

plt.figure(figsize=(11, 5))
ax = sns.barplot(x=top_actors.values, y=top_actors.index, color='#C44E52')
ax.bar_label(ax.containers[0], padding=3, fontsize=9)
plt.title('Top 10 Actors by Number of Appearances')
plt.xlabel('Number of titles')
plt.ylabel('')
plt.tight_layout()
plt.show()
```

Out [18]:



Task 7: Text & Descriptions

7(a) Most common words in the description field (word cloud)

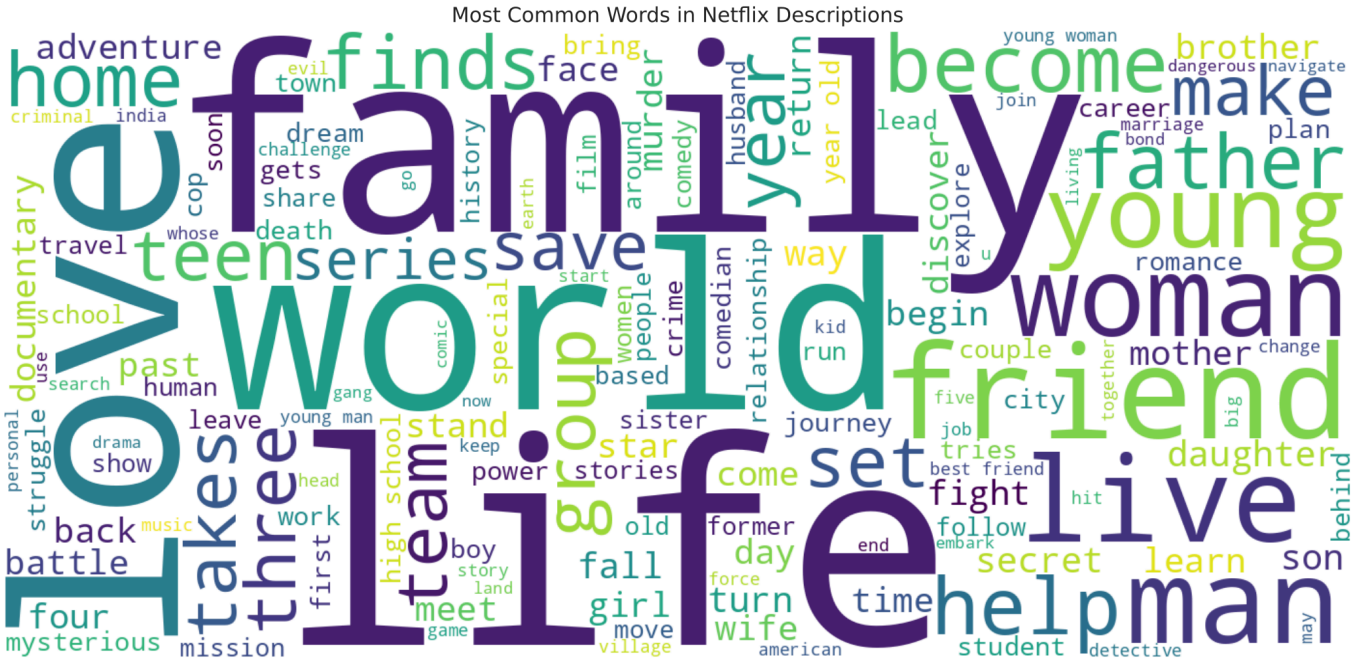
In [19]:

```
text = ' '.join(df['description'].str.lower())
stopwords = set(STOPWORDS) | {'s', 'one', 'two', 'new', 'find', 'take', 'must', 'will', 'get'}

wordcloud = WordCloud(width=1400, height=650, background_color='white', stopwords=stopwords,
                      colormap='viridis', max_words=150, random_state=42).generate(text)

plt.figure(figsize=(14, 6.5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.title('Most Common Words in Netflix Descriptions', fontsize=14)
plt.tight_layout()
plt.show()
```

Out [19]:



In [20]:

```
words = [w.strip('.,!?;:"()\'') for w in text.split()]
word_counts = Counter(w for w in words if w and w.isalpha() and w not in stopwords)
pd.DataFrame(word_counts.most_common(15), columns=['Word', 'Frequency']).set_index('Word').T
```

Out [20]:

Word	life	young	family	love	man	world	woman	friends	series	documentary	finds	school	home	three	help
Frequency	774	728	570	497	491	491	452	442	394	361	313	312	306	292	291

1 rows × 15 columns

> **Observation:** International Movies, Dramas and Comedies are the most common genres. The top directors are Rajiv Chilaka (mostly Indian children's animation) and Raúl Campos and Jan Suter (stand-up comedy specials), and the most frequent actors are mostly Indian (Anupam Kher, Shah Rukh Khan, Naseeruddin Shah) along with Japanese anime voice actors (Takahiro Sakurai, Yuki Kaji). The word cloud shows that descriptions focus on people and relationships: *life*, *young*, *family*, *love*, *man*, *world*, *woman* and *friends* are the most common words.

Task 8: In which months does Netflix add the most content?

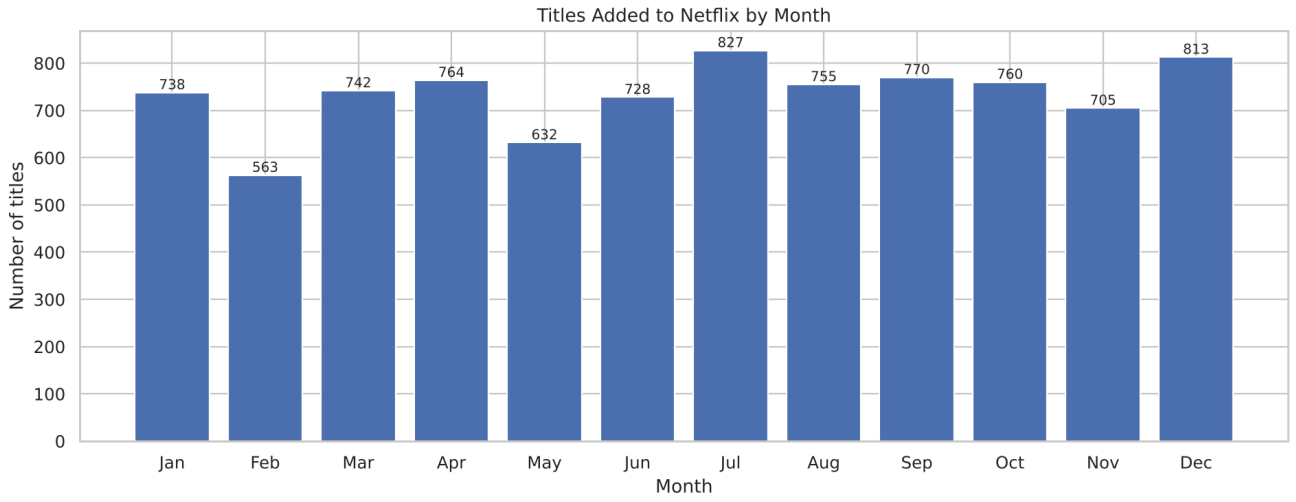
In [21]:

```
month_order = ['January', 'February', 'March', 'April', 'May', 'June', 'July',
               'August', 'September', 'October', 'November', 'December']
monthly = df['month_added'].value_counts().reindex(month_order)
print(monthly.sort_values(ascending=False).to_string())

plt.figure(figsize=(12, 4.8))
bars = plt.bar([m[:3] for m in month_order], monthly.values, color='#4C72B0')
plt.bar_label(bars, fontsize=9)
plt.title('Titles Added to Netflix by Month')
plt.xlabel('Month')
plt.ylabel('Number of titles')
plt.tight_layout()
plt.show()
```

Out [21]:

```
month_added
July          827
December      813
September     770
April         764
October       760
August        755
March         742
January       738
June          728
November      705
May           632
February      563
```



> **Observation:** Netflix adds content fairly evenly through the year. July (827 titles) and December (813) have the most additions, which matches the summer and year-end holiday seasons, while February has the fewest (563).

Task 9: Which countries produce more Movies vs TV Shows?

In [22]:

```
country_type = df[['type', 'country']].copy()
country_type = country_type[country_type['country'] != 'Unknown']
country_type['country'] = country_type['country'].str.split(',')
country_type = country_type.explode('country', ignore_index=True)
country_type['country'] = country_type['country'].str.strip()
country_type = country_type[country_type['country'] != '']

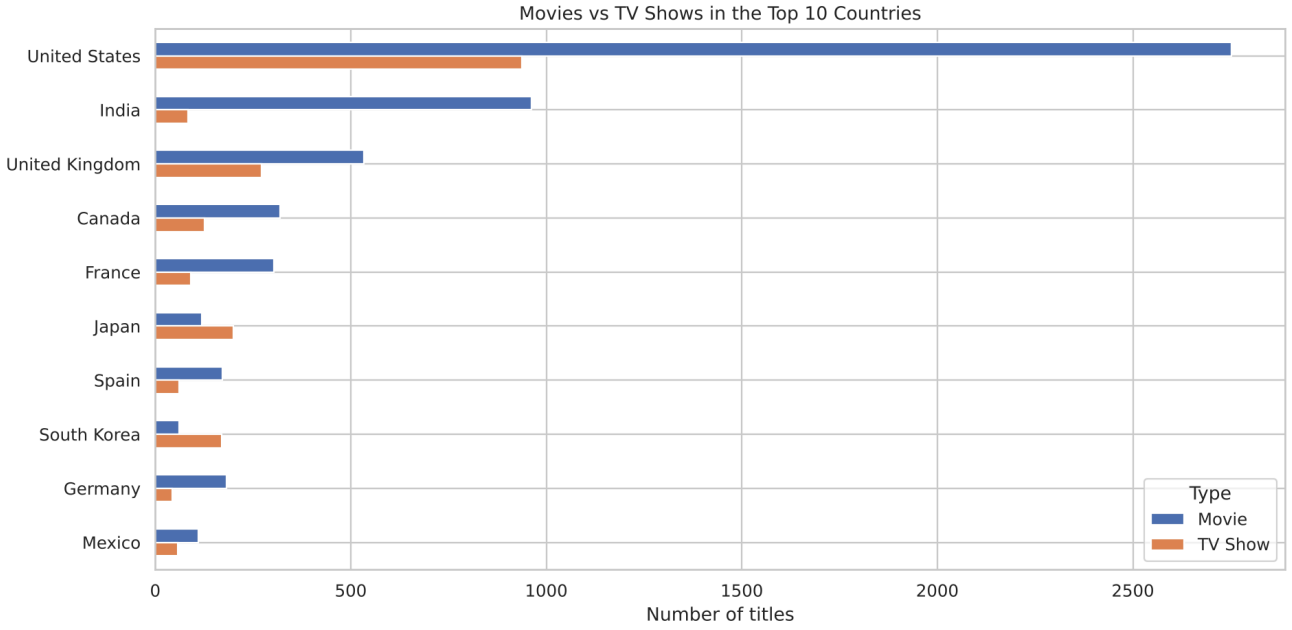
ct = pd.crosstab(country_type['country'], country_type['type'])
ct = ct.loc[ct.sum(axis=1).sort_values(ascending=False).index[:10]]
ct['Movie (%)'] = (ct['Movie'] / (ct['Movie'] + ct['TV Show']) * 100).round(1)
ct['More of'] = np.where(ct['Movie'] > ct['TV Show'], 'Movies', 'TV Shows')
display(ct)

ax = ct[['Movie', 'TV Show']].plot(kind='barh', figsize=(12, 6),
                                   color=[type_colors['Movie'], type_colors['TV Show']])
ax.invert_yaxis()
plt.title('Movies vs TV Shows in the Top 10 Countries')
plt.xlabel('Number of titles')
plt.ylabel('')
plt.legend(title='Type')
plt.tight_layout()
plt.show()
```

Out [22]:

type	Movie	TV Show	Movie (%)	More of
country				
United States	2752	938	74.6	Movies
India	962	84	92.0	Movies
United Kingdom	534	272	66.3	Movies
Canada	319	126	71.7	Movies
France	303	90	77.1	Movies
Japan	119	199	37.4	TV Shows
Spain	171	61	73.7	Movies
South Korea	61	170	26.4	TV Shows
Germany	182	44	80.5	Movies
Mexico	111	58	65.7	Movies

10 rows × 4 columns



Observation: Most countries produce more movies than TV shows. India is the most movie-focused country (about 92% movies), while South Korea and Japan are the only top-10 countries that produce more TV shows than movies, because of their popular K-dramas and anime.

Task 10: Most common ratings for Movies vs TV Shows

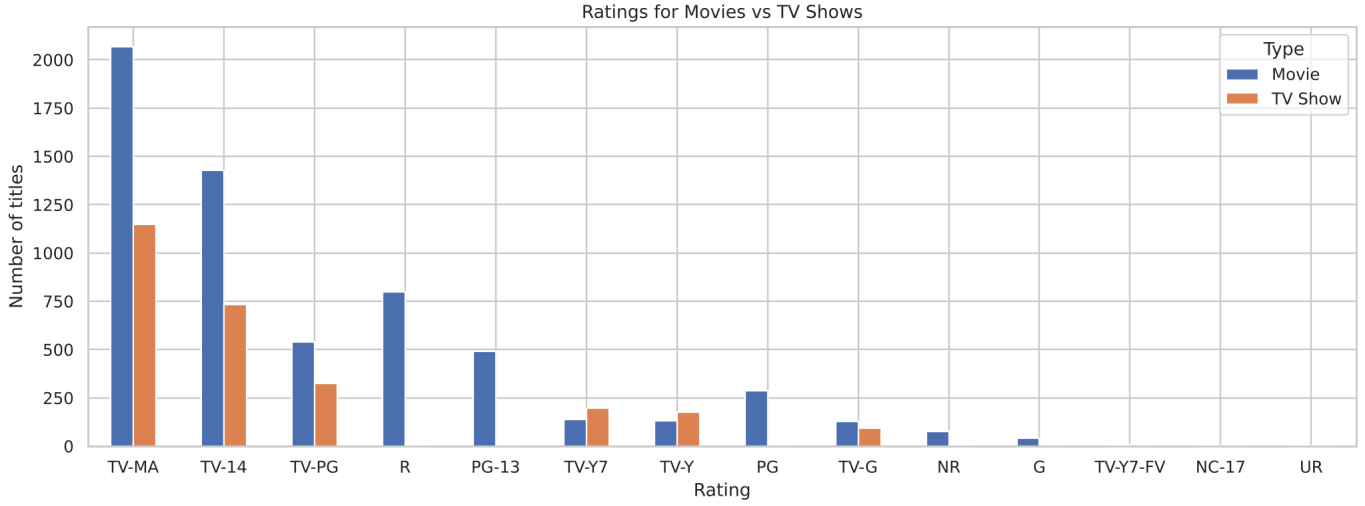
In [23]:

```
rating_type = pd.crosstab(df['rating'], df['type'])
rating_type = rating_type.loc[rating_type.sum(axis=1).sort_values(ascending=False).index]
print("Top 5 ratings for Movies  :",
rating_type['Movie'].sort_values(ascending=False).head(5).to_dict())
print("Top 5 ratings for TV Shows:", rating_type['TV
Show'].sort_values(ascending=False).head(5).to_dict())

ax = rating_type.plot(kind='bar', figsize=(13, 5), color=[type_colors['Movie'], type_colors['TV
Show']])
plt.title('Ratings for Movies vs TV Shows')
plt.xlabel('Rating')
plt.ylabel('Number of titles')
plt.xticks(rotation=0)
plt.legend(title='Type')
plt.tight_layout()
plt.show()
```

Out [23]:

```
Top 5 ratings for Movies  : {'TV-MA': 2067, 'TV-14': 1427, 'R': 797, 'TV-PG': 540, 'PG-13': 490}
Top 5 ratings for TV Shows: {'TV-MA': 1147, 'TV-14': 733, 'TV-PG': 323, 'TV-Y7': 195, 'TV-Y': 176}
```



> **Observation:** TV-MA is the most common rating for both movies and TV shows, followed by TV-14. Film ratings such as R, PG-13 and PG are used almost only for movies, while TV-Y and TV-Y7 (children's ratings) are relatively more common for TV shows.

Task 11: Oldest and newest titles on Netflix

In [24]:

```
cols = ['title', 'type', 'release_year', 'date_added', 'country']
print("Oldest titles by release year:")
display(df.sort_values('release_year').head(5)[cols])
print("Newest titles by release year:")
display(df.sort_values(['release_year', 'date_added'], ascending=False).head(5)[cols])
```

Out [24]:

Oldest titles by release year:

	title	type	release_year	date_added	country
4250	Pioneers: First Women Filmmakers*	TV Show	1925	2018-12-30	Unknown
7790	Prelude to War	Movie	1942	2017-03-31	United States
8205	The Battle of Midway	Movie	1942	2017-03-31	United States
8763	WWII: Report from the Aleutians	Movie	1943	2017-03-31	United States
8739	Why We Fight: The Battle of Russia	Movie	1943	2017-03-31	United States

5 rows × 5 columns

Newest titles by release year:

	title	type	release_year	date_added	country
1	Blood & Water	TV Show	2021	2021-09-24	South Africa
2	Ganglands	TV Show	2021	2021-09-24	Unknown
3	Jailbirds New Orleans	TV Show	2021	2021-09-24	Unknown
4	Kota Factory	TV Show	2021	2021-09-24	India
5	Midnight Mass	TV Show	2021	2021-09-24	Unknown

5 rows × 5 columns

In [25]:

```
print("First titles added to Netflix:")
display(df.sort_values('date_added').head(5)[cols])
print("Most recently added titles:")
display(df.sort_values('date_added', ascending=False).head(5)[cols])
```

Out [25]:

First titles added to Netflix:

	title	type	release_year	date_added	country
5957	To and From New York	Movie	2006	2008-01-01	United States
6611	Dinner for Five	TV Show	2007	2008-02-04	United States
5956	Just Another Love Story	Movie	2007	2009-05-05	Denmark
5955	Splatter	Movie	2009	2009-11-18	United States
7370	Mad Ron's Prevues from Hell	Movie	1987	2010-11-01	United States

5 rows × 5 columns

Most recently added titles:

	title	type	release_year	date_added	country
0	Dick Johnson Is Dead	Movie	2020	2021-09-25	United States
6	My Little Pony: A New Generation	Movie	2021	2021-09-24	Unknown
10	Vendetta: Truth, Lies and The Mafia	TV Show	2021	2021-09-24	Unknown
9	The Starling	Movie	2021	2021-09-24	United States
8	The Great British Baking Show	TV Show	2021	2021-09-24	United Kingdom

5 rows × 5 columns

> **Observation:** The oldest title is *Pioneers: First Women Filmmakers* (released 1925), followed by World War II documentaries such as *Prelude to War* and *The Battle of Midway* (1942). The newest titles were released in 2021, the last year in the dataset. The first title was added in January 2008 and the most recent additions are from 25 September 2021.

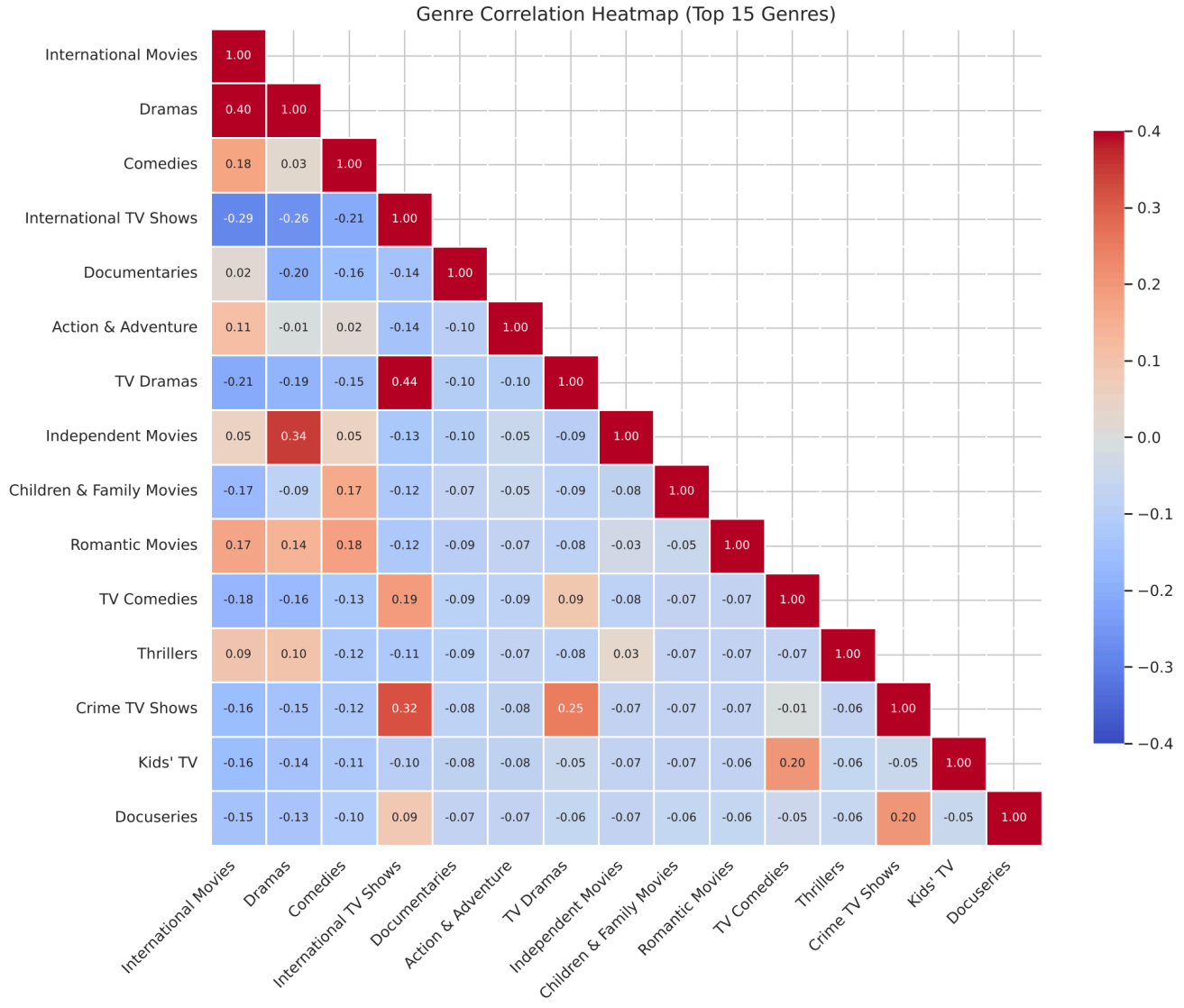
Task 12: Which genres frequently co-occur together?

In [26]:

```
# One-hot encode the genres of each title (1 if the title belongs to the genre)
genre_dummies = df['listed_in'].str.get_dummies(sep=', ')
top15 = genre_dummies.sum().sort_values(ascending=False).index[:15]
genre_corr = genre_dummies[top15].corr()

plt.figure(figsize=(13, 10.5))
mask = np.triu(np.ones_like(genre_corr, dtype=bool), k=1)
sns.heatmap(genre_corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm', vmin=-0.4, vmax=0.4,
center=0,
            linewidths=0.5, annot_kws={'size': 8.5}, cbar_kws={'shrink': 0.75})
plt.title('Genre Correlation Heatmap (Top 15 Genres)', fontsize=14)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```

Out [26]:



In [27]:

```
# Most frequent genre pairs (number of titles listed in both genres)
co_occurrence = genre_dummies.T.dot(genre_dummies)
pairs = (co_occurrence.where(np.triu(np.ones(co_occurrence.shape, dtype=bool), k=1))
         .stack().dropna().sort_values(ascending=False).head(10).astype(int))
pairs.index = [f"{a} + {b}" for a, b in pairs.index]
pairs.to_frame('Titles with both genres')
```

Out [27]:

	Titles with both genres
Dramas + International Movies	1483
Comedies + International Movies	804
Dramas + Independent Movies	588
International TV Shows + TV Dramas	514
Comedies + Dramas	502
Action & Adventure + International Movies	398
International Movies + Romantic Movies	372
International TV Shows + Romantic TV Shows	315
Dramas + Romantic Movies	308
Crime TV Shows + International TV Shows	299

10 rows × 1 columns

> **Observation:** The strongest positive correlations are between *International TV Shows* and *TV Dramas* (0.45), *International Movies* and *Dramas* (0.40), and *Dramas* and *Independent Movies* (0.34). The most frequent genre pair is *Dramas + International Movies* (1,483 titles). Negative correlations, such as *International Movies* and *International TV Shows* (−0.29), appear because movie genres and TV-show genres are never given to the same title.

Result

Thus Exploratory Data Analysis was performed on the Netflix titles dataset (8,807 titles). The analysis showed that movies make up 69.6% of the library, TV-MA is the most common rating, and the United States, India and the United Kingdom produce the most content. Netflix's additions grew rapidly after 2015 and peaked in 2019, most movies last about 87–114 minutes and most TV shows have only one season. International Movies, Dramas and Comedies are the most common genres, and the genre heatmap showed which genres frequently occur together.